Derek Lau

March 6, 2026

Foundations Of Python Programming

Assignment06

GitHubURL: https://github.com/derek-edu/IntroToProg-Python-Mod07

Object-Oriented Programming and Inheritance

Introduction

The objective of this assignment was to upgrade the student registration program by implementing an Object-Oriented Programming (OOP) architecture. This version evolves into a hierarchical class structure involving inheritance. By utilizing classes to represent related entities, the program achieves a higher level of data readability and code reusability. This approach ensures that validation logic is embedded directly within the data structures, preventing the processing and storage of invalid information.

Architectural Design via Inheritance

A ***Person*** class was defined to serve as the foundation, capturing shared attributes such as first and last names. Subsequently, a ***Student*** class was developed to inherit from the ***Person*** class, acquiring all its existing functionality while adding specific attributes for course enrollment. The use of the ***super()*** function within the ***Student*** constructor allowed the parent class to initialize shared data. This architecture allows for scalable data modeling with complex data relationships (Figure 1).

```
# TODO Create a Person Class
class Person:...


# TODO Create a Student class the inherits from the Person class
class Student(Person):
```

Figure 1: Class Inheritance between Person and Student

Data Encapsulation and Private Attributes

Data encapsulation was implemented to protect the internal state of the objects. Attributes such as name and course data were redefined as private attributes with the double underscore prefix. Access to these variables was strictly controlled through the use of ***Getters*** and ***Setters*** (Figure 2). This design choice ensures that data cannot be modified directly from outside the class, but must instead pass through a controlled interface. By centralizing and standardizing logic within the class, the risk of data corruption across different layers of the program was significantly reduced.

```python
# TODO Add first_name and last_name properties to the constructor
def __init__(self, first_name: str = '', last_name: str = ''):
    self.first_name = first_name
    self.last_name = last_name

# TODO Create a getter and setter for the first_name property
@property  # (Use this decorator for the getter or accessor)  12 usages (10 dynamic)
def first_name(self):
    return self.__first_name.title()  # formatting code

@first_name.setter  11 usages (10 dynamic)
def first_name(self, value: str):
    if value.isalpha() or value == "":  # is character or empty string
        self.__first_name = value
    else:
        raise ValueError("The first name should not contain numbers.")
```

Figure 2: Implementation of Property Getters and Setters for Private Attributes

Implementation of Validation Logic

Data integrity was further enforced by embedding validation code directly within the property setters. For the name attributes, the ***.isalpha()*** method was utilized to ensure only alphabetic characters were accepted, while the course name setter implemented a combination of ***.isalnum()*** and ***.replace()*** to allow for alphanumeric characters and spaces. Furthermore, the ***.upper()*** formatting method was introduced to ensure cleaner formatting of entered course names. To satisfy project requirements, these setters were also configured to accept empty strings, allowing for the default initialization of objects without triggering a system crash. This shift moves the responsibility of data checking from the user interface layer into the data classes themselves, providing a more resilient architecture (Figure 3).

```python
# TODO add the getter for course_name
@property  7 usages (4 dynamic)
def course_name(self):
    return self.__course_name.upper()  # keeping a consistent capitalized course name

# TODO add the setter for course_name
@course_name.setter  6 usages (5 dynamic)
def course_name(self, value: str):
    if value.replace( old: " ",  new: "").isalnum() or value == "":  # is character or empty string
    # allowing space but check for no special characters
        self.__course_name = value
    else:
        raise ValueError("Course names should only contain letters and numbers.")
```

Figure 3: Course Name Setter and Getter for Validation

Refactoring the Processing and Presentation Layers

The existing ***FileProcessor*** and ***IO*** classes were refactored to accommodate the transition from dictionaries to objects. The ***read_data_from_file*** function was updated to populate ***Student*** objects from JSON data, while the ***write_data_to_file*** function was modified to output these objects back into dictionary format for compatibility with the JSON library. This ensures that the program can easily display or export student information in a consistent, readable format (Figure 4).

```python
@staticmethod
def write_data_to_file(file_name: str, student_data: list):
    """This function writes data to a json file with data from a list of dictionary rows..."""
    file = None

    try:
        # TODO Add code to convert Student objects into dictionaries (Done)
        list_of_dictionary_data: list = []
        for student in student_data:  # Convert List of Student objects to list of dictionary rows.
            student_json: dict \
                = {"FirstName": student.first_name, "LastName": student.last_name,
                   "CourseName": student.course_name}
            list_of_dictionary_data.append(student_json)

        file = open(file_name, "w")
        json.dump(list_of_dictionary_data, file, indent=2)  # add indent

        # add file save confirmation
        print()
        print(f"Success: Data saved to {file_name}!")
        print()

        IO.output_course_names(student_data=student_data)
```

Figure 4: Conversion of Student Objects to Dictionaries for File Export

Summary

This assignment demonstrated the transition from a functional modular script to an Object-Oriented system. By implementing inheritance, the relationship between Person and Student data was formally defined and simplified. This enables future subclasses to be implemented in a scalable and controlled fashion. The use of private attributes and property setters provided a layer of data security and validation. Ultimately, this project highlighted how OOP principles such as encapsulation and inheritance contribute to the development of a reliable, scalable, and maintainable Python script.